ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN D

KIỂM THỬ TCP & HỖ TRỢ PHÂN TÍCH, XỬ LÝ SỰ CỐ

Đỗ Minh Ngọc: công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard

BIÊN SOẠN: NHÓM ĐỒ ÁN LABGUARD

PHIÊN BẢN ĐỒNG BỘ: SAU PR #11 (THÁNG 10/2026)

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN D

(Bản hướng dẫn hành động kỹ thuật tinh gọn — nắm bắt nhiệm vụ cốt lõi trong 2 phút)

1. BẠN LÀ AI?

Bạn là **Đỗ Minh Ngọc**, đảm nhiệm phân công chính thức: **Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard** (mã nhóm nhiệm vụ: LG-D-001 đến LG-D-010). Trọng tâm cốt lõi là kiểm thử socket mạng và xây dựng thư viện phân tích bằng chứng sự cố nhúng trong Admin WinForms.

2. BẠN CẦN LÀM GÌ?

- **Kiểm thử TCP thiết yếu:** Nắm chắc cơ chế Socket, đóng khung 4 byte Big-Endian (`FrameCodec`), kiểm tra lưu lượng bằng Wireshark; nghiệm thu TCP Echo (`LG-D-002` đã XONG qua PR #8) và hoàn thiện TCP Relay (`LG-D-003`) với hàng đợi Bounded Channel chống tràn RAM.
- **Phân tích bằng chứng sự cố (D-007..D-010):** Xây dựng thư viện C# thuần túy (`LabGuard.Troubleshooting`) phục vụ Admin WinForms; thẩm định chất lượng dữ liệu (độ mới, phần thiếu/cũ, trễ nhịp tim $>$ 20s); đối chiếu ma trận quy tắc tất định (Rule-Based); phân biệt rõ 3 tầng trạng thái (Kết nối, Cảnh báo, Quy trình xử lý); xếp hạng bước kiểm tra từ không xâm lấn đến can thiệp kỹ thuật; dứt khoát không đưa ra kết luận võ đoán khi thiếu bằng chứng.

3. BẠN TUYỆT ĐỐI KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo đo phần cứng máy trạm Windows hay vòng lặp Agent (việc của **Võ Trọng Phúc — A**).
- Bạn **không cần** tạo bảng SQL Server, không viết `RuleEngine` phát hiện sự cố máy chủ (việc của **Nguyễn Bá Quang — B**).
- Bạn **không cần** thiết kế giao diện WinForms Dashboard hay bắt sự kiện nút bấm UI (việc của **Lý Tuyết Vy — C**).
- *Phân định mở rộng:* Gateway L4, Collector L7 và Bộ giả lập Simulator 50 máy là **nghiên cứu mở rộng P1 tùy chọn** (nằm ở Phụ lục), không phải cam kết bắt buộc để hoàn thành nhiệm vụ chính.

4. HÔM NAY BẮT ĐẦU TỪ ĐÂU?

Đọc kỹ quy chuẩn Wire Contract (`src/LabGuard.Protocol`); nắm vững hợp đồng dữ liệu giữa Central và Admin; lập trình giao diện `ITroubleshootingEngine` nhận DTO đầu vào không phụ thuộc UI; viết bộ unit test cho 6 kịch bản sự cố chuẩn hóa (Ca 1 đến Ca 6).

5. CÁC TÀI LIỆU VÀ TẬP TIN CẦN MỞ ĐẦU TIÊN:

1. `docs/internal/TEAM_ASSIGNMENTS.md`: Phân công chính thức và ranh giới tam giác B-C-D.
2. `src/LabGuard.Protocol/FrameCodec.cs`: Cách đóng gói và giải mã tiền tố 4 byte Big-Endian.
3. `src/LabGuard.Protocol/WireEnvelope.cs`: Hợp đồng thông điệp JSON tiêu chuẩn trên mạng TCP.
4. `docs/internal/WBS_TASKS.csv`: Chi tiết tiêu chí nghiệm thu các mã LG-D-001..LG-D-010.

MỤC LỤC

Hiện trạng và phân công sau PR #11 **1**

1 Tổng quan vai trò và ranh giới kỹ thuật của Thành viên D **2**
1.1 Hiện trạng nhiệm vụ và lộ trình triển khai 2
1.2 Phân định ranh giới trách nhiệm tam giác (Quang B — Vy C — Ngọc D) 2
1.3 Bốn nguyên tắc bất biến của chức năng hỗ trợ xử lý sự cố 4

2 Nền tảng kiểm thử truyền thông TCP thiết yếu (D-001, D-002, D-003) **5**
2.1 Lập trình Socket TCP căn bản và công cụ TCP Echo (LG-D-002) 5
2.2 Quy chuẩn đóng khung 4 byte Big-Endian và phân tích bằng Wireshark 5
2.3 Chuyển tiếp TCP Relay và kiểm soát áp lực ngược Bounded Channel (LG-D-003) . 6

3 Trọng tâm D-007 đến D-010: Thư viện và thuật toán hỗ trợ phân tích sự cố **8**
3.1 Mô hình đầu vào / đầu ra và căn cứ dữ liệu (LG-D-007) 8
3.2 Thuật toán kiểm tra chất lượng dữ liệu (Độ mới, Tính đầy đủ, Phần thiếu/cũ) 9
3.3 Phân định rõ ràng 3 tầng trạng thái độc lập 9
3.4 Ma trận quy tắc giải thích và xếp hạng bước kiểm tra (LG-D-008) 10
3.5 Thiết kế kiến trúc thư viện C# `LabGuard.Troubleshooting` (LG-D-009) 12
3.6 Bộ sáu kịch bản kiểm thử tích hợp chuẩn hóa (LG-D-010) 15

4 Năm bẫy lỗi lập trình cần tránh dành cho Thành viên D **16**

5 Bộ câu hỏi vấn đáp chuyên sâu dành cho Thành viên D (20 câu) **18**

Phụ lục: Nghiên cứu mở rộng P1 tùy chọn **21**

Hiện trạng và phân công sau PR #11

TÓM TẮT 30 GIÂY

Mốc nguồn: main `70481fa`, PR #11 đã merge. Đây là cẩm nang học và kế hoạch hiện thực; các mô tả tương lai/pseudocode không phải bằng chứng tính năng đã hoạt động.

- **Agent đã có:** chương trình Console, cấu hình JSON, DeviceId riêng cho mỗi bản cài đặt, đo CPU/RAM/thời gian hoạt động Windows tại máy, chế độ thử TCP/HELLO, hủy tác vụ và ghi nhật ký.
- **Chế độ giám sát bị chặn có chủ đích** tới khi TLS/phiên kết nối/AUTH sẵn sàng. MetricsService/HeartbeatService đã có thành phần và kiểm thử nhưng chưa được bật trong chương trình khởi chạy.
- **Chưa hoàn tất:** A-005 kết nối lại, A-006 ReplayBuffer, đọc phản hồi/kiểm tra mất kết nối, tích hợp SQL/ACK và Windows Service. B-001 đã có bộ tiếp nhận TCP; B-005 mới có DDL, còn đang làm. C-001 có bố cục 0.1, chưa nghiệm thu liên kết dữ liệu và quy trình xử lý.
- **Kiểm thử đã báo cáo từ PR #11:** 195 bài đạt trên Windows tại máy: Protocol 107, Agent 64, Central 9, MockHarness 15. Đây **không phải kết quả GitHub CI**. Số 34/165 trong hồ sơ cũ là mốc lịch sử.

Thành viên	Trách nhiệm chính thức
A — Võ Trọng Phúc	Agent và Protocol.
B — Nguyễn Bá Quang	Central Server và SQL Server.
C — Lý Tuyết Vy	Admin Windows Forms.
D — Đỗ Minh Ngọc	Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard.

Nguồn: quyết định trực tiếp của người điều phối dự án ngày 17/10/2026; không tự tạo biên bản họp hoặc chữ ký. Chức năng hỗ trợ xử lý sự cố là **phần việc chính thức trong kế hoạch**, chưa phải chức năng đã được lập trình. Logic nhúng trong quy trình xử lý sự cố của LabGuard Admin, dùng dữ liệu có thẩm quyền qua Central; D phối hợp B/C, không làm ứng dụng TCP độc lập.

Gateway/Collector và Simulator là **mở rộng P1 tùy chọn** sau khi phần cốt lõi ổn định. Nội dung mở rộng được giữ để học, không phải cam kết chính của Ngọc. GK là một buổi trình bày đề xuất theo lịch giảng viên, không phải hạn hoàn thành đồ án; các chặng là kế hoạch tương đối toàn dự án.

B-009 cần đối chiếu kế hoạch: WBS cũ nhắc AssignedToUserId/IncidentHistory nhưng schema hiện chỉ có AssigneeId và chưa có bảng lịch sử. Giữ bản ghi CHƯA LÀM để Quang/nhóm quyết định, không tự sửa schema hoặc coi bảng/cột cũ đã tồn tại.

Năm mốc khác nhau: kết nối TCP; xác thực ứng dụng; nhận số liệu; ghi thành công SQL; nhận ACK ứng dụng. Chế độ thử trả exit 0 chỉ cho biết lệnh kết thúc; phía bên kia đóng kết nối có thể chưa được phát hiện. Nhật ký kết nối không chứng minh đã xác thực hoặc lưu SQL.

Bắt đầu đúng cách: đọc `docs/agent_deployment/README.md` và `docs/internal/TEAM_ASSIGNMENTS.md`. WBS giữ 34 nhiệm vụ cũ và sáu nhiệm vụ bổ sung cho chức năng hỗ trợ xử lý sự cố, đều chưa làm. Nâng cấp .NET 10 là việc riêng toàn bộ dự án, chưa thực hiện trong lượt đồng bộ này.

1. Tổng quan vai trò và ranh giới kỹ thuật của Thành viên D

TÓM TẮT 30 GIÂY

Thành viên D (**Đỗ Minh Ngọc**) phụ trách hai mảng công việc gắn kết hữu cơ: (1) **Công cụ kiểm thử truyền thông TCP** độc lập nhằm xác thực tầng mạng và điều tiết hàng đợi; (2) **Chức năng hỗ trợ phân tích, xử lý sự cố** được đóng gói thành thư viện C# nhúng trực tiếp trong LabGuard Admin. Mọi phân tích đều dựa trên bằng chứng kỹ thuật xác thực, phân định rõ ràng ranh giới với Thành viên B (Central/SQL) và Thành viên C (Admin UI).

1.1 Hiện trạng nhiệm vụ và lộ trình triển khai

Căn cứ quyết định phân công chính thức ngày 17/10/2026 và bảng WBS hợp nhất:

- **LG-D-002 (TCP Echo Console):** Đã hoàn thành và nghiệm thu qua PR #8 (chạy độc lập tại `tools/TcpEcho/Program.cs`).
- **LG-D-001 & LG-D-003 (Nền tảng TCP & TCP Relay Bounded Channel):** Đang triển khai; `tools/\path{mock_harness}` (PR #9) là công cụ bổ trợ kiểm thử, không thay thế mã nguồn chính thức của D-003.
- **LG-D-007 đến LG-D-010 (Hỗ trợ phân tích, xử lý sự cố tích hợp):** Là nhóm nhiệm vụ trọng tâm chính thức **đang ở giai đoạn thiết kế và bắt đầu lập trình (CHƯA LÀM)**. Chức năng này bắt buộc phải là một thư viện logic dùng lại được tiêu thụ bởi Admin WinForms, tuyệt đối không xây dựng thành ứng dụng chẩn đoán mạng chạy rời rạc.
- **LG-D-004, LG-D-005, LG-D-006 (Gateway, Collector, Simulator):** Là các hướng **nghiên cứu mở rộng P1 tùy chọn**, chỉ xem xét sau khi toàn bộ hệ thống cốt lõi đã nghiệm thu ổn định.

1.2 Phân định ranh giới trách nhiệm tam giác (Quang B — Vy C — Ngọc D)

Nhằm đảm bảo tính toàn vẹn kiến trúc và tránh việc chồng chéo trách nhiệm giữa các thành viên, ranh giới xử lý sự cố được quy định chặt chẽ theo Bảng 1:

Table 1: Phân định ranh giới trách nhiệm giữa Thành viên B, C và D

Thành viên	Phạm vi trách nhiệm có thẩm quyền	Điều tuyệt đối không làm
Nguyễn Bá Quang (Thành viên B)	- Lắng nghe kết nối TCP Agent, quản trị phiên kết nối. - Thực thi `RuleEngine` phát hiện sự cố dựa trên ngưỡng và hysteresis (CPU $\geq$ 90%, RAM $\geq$ 85%, nhịp tim quá hạn $>$ 20s). - Ghi nhận và lưu trữ CSDL SQL Server (`MetricSamples`, `Incidents`). - Cung cấp dữ liệu thiết bị và sự cố có thẩm quyền cho Admin.	- Không can thiệp logic giao diện hiển thị WinForms. - Không viết mã thuật toán gợi ý bước xử lý của Admin.
Đỗ Minh Ngọc (Thành viên D)	- Phát triển thư viện C# thuần túy (`LabGuard.Troubleshooting`). - Thẩm định chất lượng dữ liệu: độ mới (*freshness*), dữ liệu thiếu/cũ, trễ nhịp tim. - Đối chiếu ma trận quy tắc giải thích tất định (Rule-Based). - Xếp hạng danh mục các bước kiểm tra (ưu tiên không xâm lấn). - Dứt khoát từ chối kết luận suy đoán khi bằng chứng không đầy đủ.	- Không tự ý truy vấn trực tiếp vào CSDL SQL Server. - Không tự mở socket kết nối trực tiếp đến Agent. - Không tự ý đóng sự cố hoặc thay đổi dữ liệu Central. - Không nhúng mã giao diện WinForms vào thư viện.
Lý Tuyết Vy (Thành viên C)	- Thiết kế giao diện WinForms Admin Dashboard và bảng điều khiển sự cố. - Gọi thư viện của D để lấy nhận định và danh sách gợi ý kiểm tra. - Điều phối luồng thao tác của người quản trị (Xác nhận, Đang xử lý, Giải quyết). - Gửi lệnh cập nhật trạng thái sự cố về Central kèm kiểm soát khóa lạc quan OCC (`RowVersion`).	- Không tự ý viết mã phân tích quy tắc sự cố cứng trong WinForms Form. - Không tin cậy dữ liệu cũ khi người dùng thao tác.

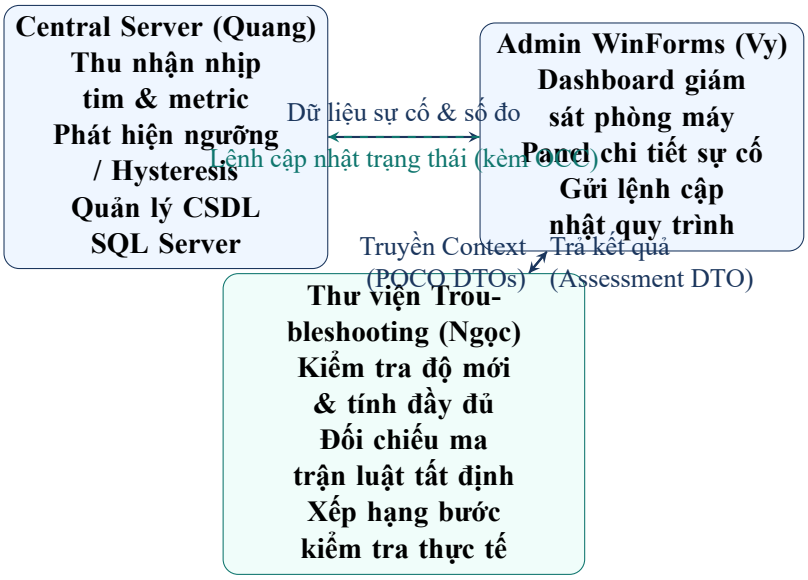


Figure 1: Sơ đồ phân định trách nhiệm tam giác B-C-D và luồng dữ liệu hỗ trợ phân tích sự cố

1.3 Bốn nguyên tắc bất biến của chức năng hỗ trợ xử lý sự cố

1. **100% Luật tất định (Deterministic Rule-Based), tuyệt đối cấm Machine Learning / Chatbot AI:** Hệ thống không sử dụng mạng nơ-ron, học máy hay gọi API mô hình ngôn ngữ lớn (LLM). Mọi gợi ý phải giải thích được chính xác dựa trên luật logic nhị phân, đảm bảo tính ổn định và khả năng tái lập trong kiểm thử.
2. **Tôn trọng hợp đồng dữ liệu thực tế:** Chỉ phân tích các trường dữ liệu thực sự có trong giao thức mạng (`WireEnvelope`) và CSDL SQL Server (`dbo.MetricSamples`, `dbo.Incidents`). Các thông tin chưa hỗ trợ (như danh sách tiến trình con `ProcessList`) bắt buộc phải ghi rõ là *Chưa có nguồn dữ liệu / Dự kiến*, tuyệt đối không bịa đặt dữ liệu mẫu.
3. **Tuyệt đối không bịa đặt điểm tin cậy ảo:** Không xuất các chỉ số phần trăm vô căn cứ (ví dụ: "khả năng lỗi CPU là 87%"). Kết luận phải là mệnh đề logic rõ ràng: *Đủ bằng chứng xác nhận*, *Nghi vấn cần kiểm chứng*, hoặc *Không đủ bằng chứng*.
4. **Dứt khoát từ chối kết luận khi thiếu dữ liệu:** Khi số mẫu metric không đủ hoặc dữ liệu nhịp tim quá hạn (> 20s), hệ thống phải xuất cảnh báo thiếu bằng chứng, dừng suy đoán và hướng dẫn kiểm tra hạ tầng mạng trước tiên.

2. Nền tảng kiểm thử truyền thông TCP thiết yếu (D-001, D-002, D-003)

TÓM TẮT 30 GIÂY

Trước khi tích hợp logic xử lý sự cố, Thành viên D phải làm chủ tầng truyền vận mạng: nắm chắc cơ chế bắt tay 3 bước TCP, lập trình socket không nghẽn, kiểm soát hiện tượng phân mảnh luồng byte bằng tiền tố độ dài 4 byte Big-Endian, và áp dụng hàng đợi Bounded Channel để điều tiết áp lực ngược (Backpressure).

2.1 Lập trình Socket TCP căn bản và công cụ TCP Echo (LG-D-002)

TCP là giao thức hướng luồng byte (*Byte Stream*), không có ranh giới bản tin tự nhiên. Khi bên gửi gọi lệnh `Write`, dữ liệu có thể bị phân mảnh (*packet fragmentation*) hoặc dồn ép (*packet coalescing*) tùy thuộc vào kích thước MSS và thuật toán Nagle của hệ điều hành.

Chương trình `TcpEcho` (PR #8) chứng minh khả năng thiết lập máy chủ phản hồi tại `tools/TcpEcho/Program.cs`:

- Lắng nghe trên cổng cục bộ bằng `TcpListener.Create(port)`.
- Tiếp nhận kết nối bất đồng bộ qua `await listener.AcceptTcpClientAsync(ct)`.
- Đọc luồng dữ liệu từ `NetworkStream` vào bộ đệm và ghi trả nguyên vẹn về máy gửi.
- **Giải phóng tài nguyên chuẩn mực:** Sử dụng cấu trúc `await using` cho cả `TcpClient` và `NetworkStream`, bắt chính xác các ngoại lệ `IOException` và `OperationCanceledException` khi đối tác ngắt kết nối đột ngột.

2.2 Quy chuẩn đóng khung 4 byte Big-Endian và phân tích bằng Wireshark

Mọi khung tin trao đổi trong LabGuard đều tuân thủ nghiêm ngặt quy cách của `src/LabGuard.Protocol/FrameCodec.cs`:

$$\text{Cấu trúc Khung Tin} = \underbrace{[\text{Length (4 bytes, Big-Endian)}]}_{\text{Tiền tố độ dài payload } N} + \underbrace{[\text{Payload JSON (UTF-8, } N \text{ bytes)}]}_{\text{Nội dung WireEnvelope}} \quad (1)$$

Khi mở Wireshark lọc gói tin bằng bộ lọc `tcp.port == 5000`, Thành viên D có thể kiểm chứng trực tiếp tính đúng đắn của từng byte nhị phân như thể hiện trong Bảng 2:

Table 2: Chi tiết phân rã byte của khung tin LabGuard trên Wireshark

Vị trí Byte	Dữ liệu Hex / Ký tự	Ý nghĩa kỹ thuật và kiểm tra chuẩn hóa
`0x0000--0x0003`	`00 00 00 B2`	4 byte độ dài Big-Endian = 178 byte payload JSON theo sau. Đọc chuẩn qua `BinaryPrimitives.ReadInt32BigEndian`.
`0x0004--0x0011`	`{"version":1,`	Phiên bản giao thức `version = 1` (số nguyên dương bắt buộc).
`0x0012--0x002D`	`"messageType":"HEARTBEAT",`	Loại bản tin quy chuẩn khớp với danh mục `MessageTypes.cs`.
`0x002E--0x005B`	`"requestId":"a1c2...-9b1a",`	Định danh UUID v4 duy nhất gồm 36 ký tự phân tách dấu gạch ngang.
`0x005C--0x007C`	`"deviceId":"AGENT-LAB01-03",`	Định danh máy trạm gửi bản tin, viết hoa chuẩn hóa.
`0x007D--0x00AE`	`"sentAtUtc":"2026-10-18T...",`	Mốc thời gian ISO 8601 chuẩn UTC, bắt buộc kết thúc bằng ký tự `Z`.
`0x00AF--0x00B5`	`"payload":{}}`	Thân gói tin JSON không chứa khóa thừa, không chứa ký tự Byte Order Mark (BOM).

2.3 Chuyển tiếp TCP Relay và kiểm soát áp lực ngược Bounded Channel (LG-D-003)

Nhiệm vụ `LG-D-003` yêu cầu xây dựng công cụ chuyển tiếp hai chiều (*Duplex Relay*) có năng lực điều tiết lưu lượng. Nếu sử dụng hàng đợi vô hạn (`Queue<T>`), khi máy chủ đích tiếp nhận chậm hoặc mạng nghẽn, bộ nhớ RAM tại trạm trung chuyển sẽ tăng không giới hạn dẫn đến sập ứng dụng (*OutOfMemoryException*).

Giải pháp bắt buộc là sử dụng `Channel.CreateBounded<byte[]>` với giới hạn dung lượng cố định (100 phần tử) và chính sách chờ `BoundedChannelFullMode.Wait`:

- **Cơ chế Áp lực ngược (Backpressure):** Khi hàng đợi đạt 100 phần tử, lệnh `await writer.WriteAsync(frame, ct)` sẽ tự động tạm dừng tác vụ ghi (*producer*) một cách bất đồng bộ. Tác vụ đọc từ socket client do đó cũng ngừng nhận dữ liệu từ card mạng, kích hoạt cơ chế TCP Flow Control (co cụm cửa sổ nhận TCP Receive Window), buộc máy gửi phải giảm tốc độ truyền tải một cách tự nhiên.

```
public sealed class BoundedTcpRelay
{
    private readonly Channel<byte[]> _relayQueue = Channel.CreateBounded<byte[]>(
        new BoundedChannelOptions(100)
        {
            FullMode = BoundedChannelFullMode.Wait,
            SingleWriter = true,
            SingleReader = true
        });

    public async Task StartRelayAsync(NetworkStream source, NetworkStream
        destination, CancellationToken ct)
    {
        // Luong 1 (Producer): Doc khung tin tu source va day vao hang doi bounded
        var readerTask = Task.Run(async () =>
        {
            while (!ct.IsCancellationRequested)
            {
```

```
                byte[]? frame = await FrameCodec.ReadFrameAsync(source, ct);
                if (frame == null) break; // Ket noi dong an toan
                await _relayQueue.Writer.WriteAsync(frame, ct); // Backpressure
                    tai day
            }
            _relayQueue.Writer.Complete();
        }, ct);

        // Luong 2 (Consumer): Lay khung tin tu hang doi va ghi toi destination
        var writerTask = Task.Run(async () =>
        {
            await foreach (var frame in _relayQueue.Reader.ReadAllAsync(ct))
            {
                await destination.WriteAsync(frame, ct);
            }
        }, ct);

        await Task.WhenAll(readerTask, writerTask);
    }
}
```

Listing 1: Mã nguồn C#: Vòng lặp chuyển tiếp dữ liệu TCP Relay áp dụng Bounded Channel

3. Trọng tâm D-007 đến D-010: Thư viện và thuật toán hỗ trợ phân tích sự cố

TÓM TẮT 30 GIÂY

Đây là nội dung kỹ thuật cốt lõi của Thành viên D. Thư viện `LabGuard.Troubleshooting` tiếp nhận ngữ cảnh sự cố từ Admin, thẩm định độ mới dữ liệu, áp dụng ma trận quy tắc đối soát tất định, phân định rạch ròi 3 tầng trạng thái và đề xuất các bước kiểm tra có thứ tự ưu tiên logic.

3.1 Mô hình đầu vào / đầu ra và căn cứ dữ liệu (LG-D-007)

Thư viện được thiết kế theo nguyên lý Clean Architecture: là một thư viện lớp C# thuần túy (*Class Library*), không phụ thuộc vào Windows Forms UI, không kết nối trực tiếp CSDL và không mở socket mạng. Mọi dữ liệu trao đổi đều qua các đối tượng DTO chuẩn hóa:

3.1.1 Cấu trúc DTO Đầu vào: `TroubleshootingContext`

- **DeviceState (Trạng thái thiết bị từ Central):**
  - `DeviceId` (string, bắt buộc): Định danh máy tính (ví dụ: `"AGENT-LAB01-03"`).
  - `MachineName` / `IpAddress` (string): Tên Windows và địa chỉ IP kết nối hiện hành.
  - `LastHeartbeatUtc` (DateTime?, bắt buộc): Thời điểm máy chủ nhận bản tin `HEARTBEAT` gần nhất.
  - `IsConnected` (bool, bắt buộc): Trạng thái kết nối socket TCP đang mở trên Central Server.
  - `AgentVersion` (string): Phiên bản Agent đang chạy.
- **IncidentRecord (Bản ghi sự cố đang chọn):**
  - `IncidentId` (long, bắt buộc): Mã sự cố định danh duy nhất.
  - `IncidentType` (string, bắt buộc): `"HIGH_CPU"`, `"HIGH_RAM"`, hoặc `"AGENT_DISCONNECTED"`.
  - `Severity` (string): `"WARNING"` hoặc `"CRITICAL"`.
  - `OpenedUtc` (DateTime, bắt buộc): Thời điểm sự cố được mở tự động bởi Central.
  - `WorkflowState` (string, bắt buộc): `"OPEN"`, `"ACKNOWLEDGED"`, `"IN_PROGRESS"`, hoặc `"RESOLVED"`.
  - `AssigneeId` (string?): Tài khoản người phụ trách xử lý (tương ứng cột `AssigneeId` trong schema).
  - `RowVersion` (byte[], bắt buộc): Khóa đồng thời OCC nhằm chống ghi đè dữ liệu.
- **RecentMetrics (Danh sách 3–10 mẫu đo gần nhất):**
  - Mỗi phần tử gồm: `CpuPercent` (float), `MemoryPercent` (float), `UptimeSeconds` (long), `TimestampUtc` (DateTime).
- **SystemTimeUtc (DateTime):** Mốc thời gian hệ thống hiện tại tại thời điểm thẩm định.

3.1.2 Cấu trúc DTO Đầu ra: `TroubleshootingAssessment`

- `DataQuality` (enum): `Fresh` (đạt chuẩn), `Stale` (quá hạn), `Insufficient` (thiếu mẫu), `Missing` (không có dữ liệu).
- `EvidenceSummary` (string): Chuỗi tóm lược bằng chứng thực tế được đối soát kèm số liệu định lượng.
- `AssessmentConclusion` (string): Nhận định kỹ thuật khách quan, nêu rõ phạm vi và giới hạn suy luận.
- `RecommendedChecks` (`IReadOnlyList<RecommendedCheck>`): Danh mục các bước kiểm tra có thứ tự ưu tiên, có cờ đánh dấu `IsNonInvasive`.
- `SuggestedActions` (`IReadOnlyList<string>`): Danh mục thao tác quản trị tương ứng trên giao diện (ví dụ: *Xác nhận tiếp nhận*, *Ghi nhận ghi chú*).

3.2 Thuật toán kiểm tra chất lượng dữ liệu (Độ mới, Tính đầy đủ, Phần thiếu/cũ)

Hệ thống không bao giờ thực hiện phân tích nếu dữ liệu không vượt qua các rào chắn kiểm định chất lượng:

1. **Kiểm tra độ mới nhịp tim (Heartbeat Freshness Gate):**

$$\Delta t_{\text{HB}} = \text{SystemTimeUtc} - \text{DeviceState.LastHeartbeatUtc} \tag{2}$$

   Nếu $\Delta t_{\text{HB}} > 20$ giây (tương đương bỏ lỡ > 4 chu kỳ nhịp tim 5s), cờ cảnh báo nhịp tim quá hạn được kích hoạt. Nếu $\Delta t_{\text{HB}} > 60$ giây, toàn bộ số liệu metric đi kèm bị coi là **STALE** (dữ liệu cũ không còn giá trị phản ánh thực trạng hiện tại).

2. **Kiểm tra tính đầy đủ của chuỗi mẫu (Metric Density Gate):** Để kết luận xu hướng tài nguyên (CPU/RAM), thuật toán yêu cầu **tối thiểu 3 mẫu liên tiếp** trong cửa sổ trượt 30 giây gần nhất. Nếu số mẫu < 3, chất lượng dữ liệu bị đánh dấu là **INSUFFICIENT**. Thư viện dứt khoát không đưa ra phán đoán tải hệ thống khi chỉ có 1 hoặc 2 mẫu rời rạc.

3. **Kiểm tra độ lệch thời gian (Clock Skew Gate):** Nếu mốc thời gian của mẫu đo nằm ở tương lai (TimestampUtc $>$ SystemTimeUtc $+$ 5s) hoặc quá xa trong quá khứ (> 300s), hệ thống phát cảnh báo lệch đồng hồ giữa Agent và Server.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Nguyên tắc chặn đứng suy đoán: Khi `DataQuality != DataQualityStatus.Fresh`, hàm phân tích lập tức ngắt chuỗi suy luận, trả về thông báo: *"Dữ liệu đo đạc quá hạn hoặc không đủ số mẫu quan trắc (chỉ có N/3 mẫu). Hệ thống bảo lưu kết luận chẩn đoán. Khuyến nghị kiểm tra kết nối mạng và dịch vụ Agent trước khi thao tác kỹ thuật."*

3.3 Phân định rõ ràng 3 tầng trạng thái độc lập

Một trong những lỗi nghiêm trọng nhất là gộp chung hoặc nhầm lẫn giữa 3 tầng trạng thái trong LabGuard:

Table 3: So sánh bản chất và chu kỳ vòng đời của 3 tầng trạng thái trong LabGuard

Tầng trạng thái	Khái niệm kỹ thuật	Các giá trị trạng thái	Cơ chế kích hoạt & chuyển đổi
Tầng 1: Kết nối mạng (Connection)	Tình trạng kênh truyền vật lý/socket giữa Agent và Central Server.	- `CONNECTED` - `DISCONNECTED` - `HEARTBEAT_TIMEOUT`	Bắt tay TCP socket hoặc vòng lặp kiểm tra nhịp tim định kỳ 5s. Quá 20s không có tin báo thì chuyển sang Disconnected.
Tầng 2: Điều kiện cảnh báo (Alert Condition)	Tình trạng vượt ngưỡng tài nguyên tại máy trạm, do Central tính toán.	- `TRIGGERED` - `RECOVERED` - `NORMAL`	Tính toán theo thuật toán Hysteresis: - CPU $\geq$ 90% (3 mẫu liên tiếp) $\rightarrow$ Triggered. - CPU $\leq$ 75% (3 mẫu liên tiếp) $\rightarrow$ Recovered.
Tầng 3: Quy trình xử lý (Incident Workflow)	Vòng đời sự cố nghiệp vụ lưu trữ bền vững trong bảng `dbo.Incidents`.	- `OPEN` - `ACKNOWLEDGED` - `IN_PROGRESS` - `RESOLVED`	Do con người (Quản trị viên) thao tác chuyển trạng thái qua Admin WinForms, xác thực khóa lạc quan OCC.

GHI CHÚ QUAN TRỌNG

LƯU Ý CỐT TỬ VỀ ĐIỀU KIỆN PHỤC HỒI (RECOVERED): Khi tài nguyên hạ nhiệt (ví dụ CPU giảm từ 95% xuống 40%), Tầng 2 ghi nhận điều kiện là **RECOVERED**. Tuy nhiên, sự kiện này **TUYỆT ĐỐI KHÔNG TỰ ĐỘNG ĐÓNG SỰ CỐ SANG RESOLVED** ở Tầng 3. Sự cố vẫn giữ nguyên trạng thái (`OPEN` hoặc `IN_PROGRESS`) trên giao diện Admin cho đến khi người quản trị vào xem xét lịch sử, xác nhận nguyên nhân và chủ động nhấn nút giải quyết.

3.4 Ma trận quy tắc giải thích và xếp hạng bước kiểm tra (LG-D-008)

Thư viện áp dụng ma trận đối soát quy tắc logic nhị phân được mô tả tại Bảng 4:

Table 4: Ma trận quy tắc phân tích bằng chứng kỹ thuật (Rule-Based Matrix)

Tình huống sự cố	Bằng chứng quan sát được	Nhận định kỹ thuật của D	Xếp hạng bước kiểm tra gợi ý
CPU cao kéo dài (Sustained CPU)	- 3 mẫu liên tiếp: CPU $\geq$ 90%. - Socket TCP: Đang kết nối. - Nhịp tim: Mới ($<$ 10s).	Tải tính toán cao liên tục, thỏa mãn điều kiện mở sự cố `HIGH_CPU`. Không xác định đích danh tiến trình do thiếu dữ liệu process.	1. (*Không xâm lấn*) Hỏi sinh viên đang chạy tác vụ nặng (biên dịch mã, render đồ họa). 2. (*Quan sát*) Mở Task Manager trực tiếp tại máy trạm kiểm tra tiến trình chiếm dụng. 3. (*Can thiệp*) Yêu cầu dừng tiến trình nếu không thuộc bài thực hành.
Đột biến CPU (Transient Spike)	- Mẫu 1: CPU = 96%. - Mẫu 2–3: CPU = 65%, 58%. - Socket TCP: Đang kết nối.	Tải tăng đột biến cục bộ trong thời gian ngắn, sau đó tự hồi phục về ngưỡng an toàn. Không phải sự cố suy giảm tài nguyên nghiêm trọng.	1. (*Không xâm lấn*) Tiếp tục theo dõi chuỗi số liệu tiếp theo. 2. (*Ghi nhận*) Không cần can thiệp kỹ thuật hay khởi động lại máy.
Tràn bộ nhớ RAM (Memory Leak)	- 3 mẫu liên tiếp: RAM $\geq$ 85%. - CPU: Ổn định ở mức thấp ($<$ 40%). - Socket: Kết nối bình thường.	Bộ nhớ cạn kiệt kéo dài, có nguy cơ gây tràn bộ nhớ ảo (Pagefile thrashing) và treo ứng dụng người dùng.	1. (*Không xâm lấn*) Đối chiếu thời gian hoạt động (`UptimeSeconds`). 2. (*Quan sát*) Kiểm tra danh sách ứng dụng nền chưa giải phóng RAM. 3. (*Can thiệp*) Đóng các tab trình duyệt hoặc ứng dụng rò rỉ RAM.
Mất kết nối đột ngột (Sudden Drop)	- Socket TCP: Đã ngắt. - Mất nhịp tim: $>$ 20 giây. - Mẫu CPU/RAM trước khi ngắt: Bình thường ($<$ 50%).	Mất liên lạc mạng ở tầng truyền vận. Nguyên nhân nghiêng về sự cố hạ tầng kết nối vật lý, không phải do quá tải tài nguyên gây crash.	1. (*Không xâm lấn*) Ping kiểm tra địa chỉ IP máy trạm từ Admin. 2. (*Vật lý*) Kiểm tra đèn LED cáp mạng LAN hoặc Switch phòng máy. 3. (*Dịch vụ*) Kiểm tra người dùng có vô tình tắt máy hoặc ngắt card mạng.
Sự cố kép: Quá tải dẫn đến sập kết nối	- 3 mẫu đo cuối: CPU $\geq$ 98%, RAM $\geq$ 92%. - Sau đó: Mất nhịp tim $>$ 25s, ngắt socket TCP.	Hiện tượng quá tải cực độ dẫn đến hệ điều hành bị đóng băng hoặc tiến trình Agent bị Crash do cạn kiệt tài nguyên hệ thống.	1. (*Quan sát*) Quan sát màn hình máy trạm xem có hiện tượng màn hình xanh (BSOD) hoặc đơ cứng chuột. 2. (*Can thiệp*) Khởi động lại máy tính bằng nút nguồn vật lý nếu máy mất phản hồi hoàn toàn.
Bằng chứng thiếu / Dữ liệu quá hạn	- Nhịp tim cuối: Cách đây $>$ 90s. - Số mẫu metric: Chỉ có 0 hoặc 1 mẫu. - Sự cố trên bảng: Đang `OPEN`.	Dữ liệu đo đạc quá cũ, thiếu căn cứ lượng hóa để đưa ra kết luận về hiện trạng tải của máy trạm.	1. (*Không xâm lấn*) Không kết luận máy đang quá tải hay bình thường. 2. (*Hạ tầng*) Kiểm tra luồng gửi metric từ Agent tới Central. 3. (*Khởi động lại*) Khởi động lại dịch vụ Agent để khôi phục thu thập.

3.4.1 Thuật toán xếp hạng bước kiểm tra (Ranked Inspection Steps Algorithm)

Các bước kiểm tra trong `RecommendedChecks` luôn tuân thủ nguyên tắc:

1. **Mức ưu tiên 1 (Không xâm lấn — Non-Invasive):** Thu thập thêm thông tin quan trắc (Ping, xem log, đối chiếu tác vụ bài học) mà không làm gián đoạn sinh viên đang sử dụng máy.
2. **Mức ưu tiên 2 (Tác động thấp — Low-Impact):** Kiểm tra các dịch vụ nền, xem Task Manager, kiểm tra thiết bị phần cứng lân cận.
3. **Mức ưu tiên 3 (Can thiệp xâm lấn — Invasive/High-Impact):** Buộc dừng tiến trình, khởi động lại dịch vụ Agent hoặc khởi động lại máy trạm. Bước này chỉ được đề xuất khi các bằng chứng ở Mức 1 và 2 khẳng định máy đã mất khả năng tự phục hồi.

3.5 Thiết kế kiến trúc thư viện C# `LabGuard.Troubleshooting` (LG-D-009)

Mã nguồn thư viện được thiết kế độc lập, sáng sủa và dễ bảo trì:

```
namespace LabGuard.Troubleshooting;

public enum DataQualityStatus
{
    Fresh,
    Stale,
    Insufficient,
    Missing
}

public sealed record RecommendedCheck(
    int StepNumber,
    string Title,
    string Description,
    string Priority,
    bool IsNonInvasive
);

public sealed record TroubleshootingAssessment(
    DataQualityStatus DataQuality,
    string EvidenceSummary,
    string AssessmentConclusion,
    IReadOnlyList<RecommendedCheck> RecommendedChecks,
    IReadOnlyList<string> SuggestedActions
);

public interface ITroubleshootingEngine
{
    TroubleshootingAssessment Analyze(TroubleshootingContext context);
}
```

Listing 2: Giao diện và mô hình dữ liệu của Troubleshooting Engine

```
public sealed class TroubleshootingEngine : ITroubleshootingEngine
{
    private const float CpuThreshold = 90.0f;
    private const float RamThreshold = 85.0f;
    private const double HeartbeatStaleSeconds = 20.0;

    public TroubleshootingAssessment Analyze(TroubleshootingContext ctx)
    {
        // 1. Kiem tra tinh san co cua du lieu
```

```
        if (ctx.Device == null || ctx.Incident == null)
        {
            return new TroubleshootingAssessment(
                DataQualityStatus.Missing,
                "Khong tim thay thong tin thiet bi hoac ban ghi su co.",
                "Thieu nguon du lieu hop le tu Central.",
                Array.Empty<RecommendedCheck>(),
                new[] { "Yeu cau dong bo lai du lieu tu Central" });
        }

        // 2. Kiem tra do moi cua nhip tim
        var hbAge = ctx.Device.LastHeartbeatUtc.HasValue
            ? (ctx.SystemTimeUtc - ctx.Device.LastHeartbeatUtc.Value).TotalSeconds
            : double.MaxValue;

        var isHeartbeatStale = hbAge > HeartbeatStaleSeconds;

        // 3. Kiem tra mat do chuoi mau metrics
        var samples = ctx.RecentMetrics?.OrderByDescending(m =>
            m.TimestampUtc).Take(3).ToList()
                      ?? new List<MetricSampleDto>();

        if (samples.Count < 3)
        {
            return new TroubleshootingAssessment(
                DataQualityStatus.Insufficient,
                $"Chi thu thap duoc {samples.Count}/3 mau metric gan nhat (Nhip
                    tim: {hbAge:F1}s truoc).",
                "Chua du mau quan trac lien tiep de khang dinh xu huong tai
                    nguyen. He thong bao luu ket luan.",
                new[]
                {
                    new RecommendedCheck(1, "Kiem tra luong telemetry", "Xac minh
                        Agent co gui du MetricBatch 10s/lan khong.", "HIGH", true),
                    new RecommendedCheck(2, "Theo doi tiep tuc", "Cho them 2 chu
                        ky thu thap de du bo 3 mau doi chieu.", "MEDIUM", true)
                },
                new[] { "Giu nguyen su co o trang thai OPEN, cho cap nhat" });
        }

        // 4. Neu du lieu da cu tren 60s, bao STALE
        if (hbAge > 60.0)
        {
            return new TroubleshootingAssessment(
                DataQualityStatus.Stale,
                $"So lieu da cu ({hbAge:F0}s truoc). Mau CPU cuoi cung:
                    {samples[0].CpuPercent:F1}%.",
                "Du lieu quan trac da qua han. Khong dua ra ket luan suy doan ve
                    tai hien tai.",
                new[]
                {
                    new RecommendedCheck(1, "Kiem tra ket noi vat ly", "Kiem tra
                        cap mang LAN tai may tram.", "HIGH", true),
                    new RecommendedCheck(2, "Kiem tra tien trinh Agent", "Xac
                        minh ung dung Agent con hoat dong tren may tram khong.",
                        "MEDIUM", false)
                },
                new[] { "Xac nhan su co tiep nhan xu ly" });
        }

```

```
        // 5. Doi soat ma tran quy tac khi du lieu FRESH
        var avgCpu = samples.Average(s => s.CpuPercent);
        var isSustainedCpu = samples.All(s => s.CpuPercent >= CpuThreshold);
        var isSustainedRam = samples.All(s => s.MemoryPercent >= RamThreshold);

        if (ctx.Incident.IncidentType == "HIGH_CPU" && isSustainedCpu)
        {
            return new TroubleshootingAssessment(
                DataQualityStatus.Fresh,
                $"3 mau CPU lien tiep vuot nguong: {string.Join("%, ",
                    samples.Select(s => s.CpuPercent.ToString("F1")))}% (Trung
                    binh: {avgCpu:F1}%).",
                "Xac nhan tinh trang qua tai CPU keo dai tren 30 giay. Phu hop
                    voi canh bao HIGH_CPU.",
                new[]
                {
                    new RecommendedCheck(1, "Doi chieu bai thuc hanh", "Kiem tra
                        sinh vien co dang chay tac vu nang duoc phep khong.",
                        "NORMAL", true),
                    new RecommendedCheck(2, "Quan sat Task Manager", "Mo Task
                        Manager truc tiep xem tien trinh chiem CPU cao nhat.",
                        "HIGH", true),
                    new RecommendedCheck(3, "Can thiep tien trinh", "Dong tien
                        trinh chay ngam bat thuong neu can thiet.", "URGENT",
                        false)
                },
                new[] { "Chuyen trang thai sang IN_PROGRESS", "Ghi chu nguyen
                    nhan xu ly" });
        }

        // Xu ly cac quy tac khac tuong tu...
        return BuildDefaultAssessment(ctx, samples, hbAge);
    }
}
```

Listing 3: Mã nguồn C#: Cốt lõi xử lý logic thẩm định bằng chứng

3.6 Bộ sáu kịch bản kiểm thử tích hợp chuẩn hóa (LG-D-010)

Để chuẩn bị cho việc nghiệm thu tích hợp cùng Thành viên B và C, Thành viên D chuẩn bị sẵn bộ 6 kịch bản kiểm thử chuẩn hóa với đầy đủ dữ liệu giả định và kết quả kỳ vọng được mô tả chi tiết tại Bảng 5:

Table 5: Bộ 6 kịch bản kiểm thử tích hợp của Thư viện Troubleshooting (Ca 1 đến Ca 6)

Mã Ca	Tình huống giả định	Dữ liệu đầu vào (Context)	Đánh giá Chất lượng	Kết luận nhận định kỹ thuật	Các bước kiểm tra đề xuất (Ưu tiên)
Ca 1	Quá tải CPU kéo dài trên máy `MAY-03`.	- CPU: 91.2% → 93.1% → 92.4% - RAM: 58.5% - TCP: Connected, HB: 4s - Sự cố: `HIGH_CPU`, `OPEN`	**Fresh** (Đủ 3 mẫu, nhịp tim hợp lệ).	Xác nhận quá tải CPU liên tục > 30s. Phù hợp cảnh báo mở. Giới hạn: chưa có danh sách process.	1. Hỏi sinh viên về tác vụ đồ họa/mã nguồn. 2. Mở Task Manager tại máy kiểm tra process. 3. Đóng tiến trình nếu bất thường.
Ca 2	Đột biến CPU nhất thời trên máy `MAY-07`.	- CPU: 96.5% → 62.1% → 58.4% - RAM: 45.0% - TCP: Connected, HB: 3s - Chưa có sự cố mở	**Fresh** (Đủ 3 mẫu, nhịp tim hợp lệ).	Đột biến CPU tức thời trong 1 mẫu rồi tự hạ dưới 75%. Không thỏa điều kiện mở cảnh báo.	1. Tiếp tục giám sát chu kỳ tiếp theo. 2. Không can thiệp kỹ thuật.
Ca 3	Tràn bộ nhớ RAM kéo dài trên máy `MAY-12`.	- RAM: 88.4% → 89.1% → 91.0% - CPU: 24.5% - TCP: Connected, HB: 5s - Sự cố: `HIGH_RAM`, `OPEN`	**Fresh** (Đủ 3 mẫu, nhịp tim hợp lệ).	Xác nhận cạn kiệt RAM liên tục. Nguy cơ nghẽn ảo hóa bộ nhớ. Phù hợp cảnh báo `HIGH_RAM`.	1. Xem Task Manager rà soát tiến trình ngốn RAM. 2. Đóng bớt tab trình duyệt / ứng dụng rò rỉ RAM.
Ca 4	Mất kết nối đột ngột trên máy `MAY-15`.	- Socket TCP: Đã ngắt - HB: Quá hạn 26s - CPU trước đó: 14.2% - Sự cố: `AGENT_DISCONNECTED`	**Stale / Disconnected** (Mất tín hiệu nhịp tim).	Mất liên lạc mạng ở tầng socket. Tải trước khi mất ở mức thấp, nghi vấn sự cố cáp hoặc tắt máy.	1. Ping IP máy trạm từ Admin. 2. Kiểm tra cáp mạng LAN phòng máy. 3. Kiểm tra công tắc nguồn máy trạm.
Ca 5	Sự cố kép: Quá tải dẫn đến sập máy `MAY-09`.	- 3 mẫu cuối: CPU $\geq$ 98%, RAM $\geq$ 94% - Sau đó: Mất HB > 28s, TCP đứt - Sự cố: `AGENT_DISCONNECTED`	**Critical Transition** (Chuyển từ tải cao sang mất tín hiệu).	Hiện tượng quá tải cực độ ngay trước khi mất kết nối. Khả năng cao máy bị treo cứng hoặc Agent bị crash.	1. Quan sát trực tiếp màn hình máy trạm. 2. Kiểm tra log hệ thống Event Viewer. 3. Bấm nút Reset cứng máy trạm nếu treo hoàn toàn.
Ca 6	Dữ liệu thiếu / Quá hạn trên máy `MAY-02`.	- Sự cố: `HIGH_CPU` đang `OPEN` - Chỉ có 1 mẫu CPU (92.0%) - Nhịp tim cuối: Cách đây 110s	**Insufficient & Stale** (Thiếu số mẫu, quá hạn nặng).	Dữ liệu đo đạc quá hạn và thiếu mẫu. Hệ thống từ chối kết luận suy đoán về hiện trạng tải của máy.	1. Kiểm tra luồng gửi telemetry của Agent. 2. Khởi động lại dịch vụ Agent trên máy trạm. 3. Không đóng hay sửa mức cảnh báo sự cố.

4. Năm bẫy lỗi lập trình cần tránh dành cho Thành viên D

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Đồng nhất một lần đọc TCP (`NetworkStream.ReadAsync`) với một thông điệp hoàn chỉnh
Bản chất: TCP là giao thức dòng byte không ranh giới. Một bản tin JSON 500 byte có thể bị chia thành hai lần nhận (200 byte và 300 byte), hoặc ba bản tin nhịp tim liên tiếp bị gom lại trong một bộ đệm đọc.
Cách phòng tránh: Tuyệt đối không parse trực tiếp JSON từ một lệnh `ReadAsync`. Bắt buộc phải đọc đủ 4 byte Big-Endian tiền tố bằng `FrameCodec.ReadExactlyBytesAsync`, lấy đúng kích thước N, sau đó đọc chính xác tiếp N byte payload rồi mới tiến hành giải mã JSON.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Phán đoán nguyên nhân võ đoán hoặc bịa đặt điểm tin cậy ảo khi dữ liệu thiếu/cũ
Bản chất: Khi thấy sự cố CPU mà nhịp tim đã mất hơn 60 giây, lập trình viên vội vã in ra thông báo: "Máy tính đang bị quá tải 95%". Đây là lỗi logic học thuật nghiêm trọng vì dữ liệu đã mất tính thời sự.
Cách phòng tránh: Tuân thủ nghiêm ngặt rào chắn kiểm tra chất lượng dữ liệu (*Data Quality Gate*). Nếu thiếu mẫu (< 3 mẫu) hoặc nhịp tim trễ (> 20s), dứt khoát trả về trạng thái `Insufficient` hoặc `Stale`, từ chối đưa ra kết luận chẩn đoán sâu và hướng dẫn người dùng kiểm tra đường truyền trước.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Nhầm lẫn giữa điều kiện phục hồi (Alert Hysteresis) với việc đóng sự cố (Workflow)
Bản chất: Khi nhận được 3 mẫu CPU $\leq 75\%$, lập trình viên tự động cập nhật trạng thái sự cố trong CSDL thành `RESOLVED`. Điều này phá hủy quy trình kiểm soát của quản trị viên và làm mất vết lịch sử sự cố chưa được kiểm tra thực địa.
Cách phòng tránh: Phân biệt rạch ròi: `RECOVERED` là trạng thái của Tầng 2 (Điều kiện cảnh báo), trong khi `RESOLVED` thuộc Tầng 3 (Quy trình nghiệp vụ). Điều kiện phục hồi chỉ mang ý nghĩa thông báo cho quản trị viên biết tải đã hạ nhiệt; chỉ có con người mới có quyền xác nhận đóng sự cố.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Thư viện phân tích vi phạm ranh giới, tự ý kết nối SQL hoặc nhúng mã WinForms
Bản chất: Để tiện lấy dữ liệu, thư viện của D tự mở kết nối `SqlConnection` truy vấn bảng `dbo.Incidents`, hoặc trực tiếp thay đổi màu sắc của các Control trên WinForms. Điều này biến mã nguồn thành một mớ hỗn độn (Spaghetti Code), gây xung đột đa luồng và không thể viết Unit Test độc lập.
Cách phòng tránh: Đóng gói thư viện phân tích thành Pure C# Library. Toàn bộ dữ liệu đầu vào được truyền qua đối tượng ngữ cảnh (`TroubleshootingContext`), kết quả trả về là DTO thuần túy. Việc đọc DB do Thành viên B đảm trách, việc hiển thị WinForms do Thành viên C đảm trách.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Rò rỉ tài nguyên Socket và nghẽn bộ nhớ do sử dụng hàng đợi không giới hạn
Bản chất: Trong các công cụ TCP Relay hoặc kiểm thử, sử dụng `Queue<T>` không giới hạn làm bộ nhớ phình to khi đối tác xử lý chậm. Đồng thời, không đóng socket đúng cách khi client ngắt kết nối khiến cổng socket bị treo ở trạng thái `CLOSE_WAIT`.
Cách phòng tránh: Luôn sử dụng `Channel.CreateBounded` với cơ chế `BoundedChannelFullMode.Wait` để áp dụng áp lực ngược. Đặt mọi socket và stream trong khối `await using`, bắt kịp thời các ngoại lệ `IOException` và hủy bỏ `CancellationToken` một cách dứt khoát.

5. Bộ câu hỏi vấn đáp chuyên sâu dành cho Thành viên D (20 câu)

TÓM TẮT 30 GIÂY

Bộ câu hỏi được phân bổ cân bằng phản ánh chính xác phân công thực tế: 8 câu về nền tảng mạng TCP và kiểm thử dòng byte; 10 câu về thuật toán hỗ trợ phân tích sự cố và ranh giới kiến trúc tam giác; 2 câu về các nghiên cứu mở rộng P1 tùy chọn.

1. **Tại sao giao thức TCP lại đòi hỏi phải có cơ chế đóng khung (Framing) như tiền tố 4 byte Big-Endian?**
   Trả lời: Vì TCP là giao thức hướng dòng byte (*Byte Stream*), tầng truyền vận không bảo lưu ranh giới giữa các lần gọi gửi tin. Tiền tố 4 byte Big-Endian chỉ rõ độ dài chính xác của gói tin tiếp theo, giúp bên nhận gom đủ các mẩu byte bị phân mảnh trước khi tiến hành giải mã JSON, ngăn ngừa lỗi vỡ khung dữ liệu.

2. **Cơ chế Big-Endian hoạt động ra sao và hiển thị thế nào trên Wireshark?**
   Trả lời: Big-Endian lưu trữ byte có trọng số lớn nhất ở địa chỉ bộ nhớ thấp nhất (thứ tự mạng chuẩn). Ví dụ một khung tin dài 178 byte ($0x000000B2$) sẽ xuất hiện trên cửa sổ Byte Viewer của Wireshark đúng thứ tự 4 byte liên tiếp: `00 00 00 B2`.

3. **Chương trình TCP Echo (`LG-D-002`) giải quyết bài toán kỹ thuật gì trong đồ án?**
   Trả lời: TCP Echo kiểm chứng khả năng thiết lập socket lắng nghe, tiếp nhận kết nối đa luồng, đọc ghi qua `NetworkStream` và xử lý giải phóng tài nguyên an toàn khi client ngắt kết nối đột ngột, làm tiền đề tin cậy cho các phân hệ mạng phức tạp hơn.

4. **Tại sao hàng đợi trong TCP Relay (`LG-D-003`) bắt buộc phải là Bounded Channel?**
   Trả lời: Để ngăn chặn nguy cơ cạn kiệt bộ nhớ RAM khi tốc độ gửi của client lớn hơn tốc độ nhận của server đích. Hàng đợi giới hạn 100 phần tử kích hoạt cơ chế áp lực ngược (*Back-pressure*) với chính sách `FullMode.Wait`, buộc client phải tạm dừng gửi khi hàng đợi bị đầy.

5. **Làm thế nào để bắt chính xác sự kiện đối tác ngắt kết nối TCP đột ngột trong C#?**
   Trả lời: Khi đối tác đóng kết nối, lệnh `ReadAsync` trên `NetworkStream` sẽ trả về giá trị 0 (tương đương ký hiệu End-Of-Stream). Nếu đối tác bị rút cáp hoặc sập nguồn, socket sẽ ném ngoại lệ `IOException` hoặc `SocketException`. Bắt hai trường hợp này cho phép đóng kết nối an toàn.

6. **Bộ lọc Wireshark `tcp.port == 5000` bắt được những giai đoạn nào của phiên truyền thông?**
   Trả lời: Bắt được toàn bộ vòng đời phiên: bắt tay 3 bước (`SYN`, `SYN-ACK`, `ACK`), các khung tin dữ liệu kèm cờ `PSH, ACK`, bản tin nhịp tim định kỳ, và quá trình đóng kết nối an toàn (`FIN, ACK`) hoặc ngắt cưỡng bức (`RST`).

7. **Sự khác biệt giữa việc kiểm thử bằng stub cục bộ (`mock_harness`) và kiểm thử tích hợp mạng thật là gì?**
   Trả lời: Stub cục bộ kiểm chứng tính toàn vẹn cú pháp của hợp đồng dữ liệu trong môi trường đơn máy không trễ; kiểm thử mạng thật trên cổng 5000 đối mặt với độ trễ LAN, hiện tượng phân mảnh byte, tranh chấp kết nối đồng thời và rủi ro đứt cáp vật lý.

8. **Tại sao chu kỳ nhịp tim là 5 giây nhưng thời gian xác định mất kết nối lại là > 20 giây?**
   Trả lời: Để thiết lập khoảng dung sai an toàn chống báo động giả (*False Positive*) khi mạng

phòng máy bị suy giảm chất lượng hoặc trễ tức thời. Ngưỡng 20 giây tương đương việc bỏ lỡ liên tiếp 4 bản tin nhịp tim.

9. **Vai trò chính thức của Thành viên D trong chức năng hỗ trợ phân tích sự cố là gì?**
   Trả lời: Xây dựng thư viện C# thuần túy (`LabGuard.Troubleshooting`) để thẩm định chất lượng bằng chứng, đối soát ma trận quy tắc giải thích tất định và xếp hạng các bước kiểm tra hỗ trợ quản trị viên trong Admin WinForms.

10. **Tại sao hệ thống hỗ trợ xử lý sự cố tuyệt đối không dùng Machine Learning hay Chatbot AI?**
    Trả lời: Vì trong giám sát phòng máy, quản trị viên yêu cầu tính giải thích minh bạch, độ tin cậy tuyệt đối và tính tất định. Mô hình học máy dạng hộp đen hoặc chatbot có thể tạo ra ảo giác (hallucination), suy đoán sai lệch và không thể kiểm chứng bằng unit test tự động.

11. **Thư viện của D lấy dữ liệu từ đâu và có được phép truy vấn trực tiếp SQL Server không?**
    Trả lời: Tuyệt đối không được truy vấn SQL Server trực tiếp. Thư viện chỉ nhận dữ liệu có thẩm quyền từ Central Server được chuyển tiếp qua Admin thông qua đối tượng ngữ cảnh `TroubleshootingContext`.

12. **Thế nào là dữ liệu STALE và thư viện xử lý tình huống này ra sao?**
    Trả lời: Dữ liệu STALE là số liệu đo đạc có mốc thời gian quá cũ (> 60s) hoặc nhịp tim trễ quá 20s. Khi gặp dữ liệu STALE, thư viện lập tức ngắt phân tích sâu, từ chối kết luận nguyên nhân và hướng dẫn quản trị viên kiểm tra đường truyền kết nối trước.

13. **Tại sao thuật toán yêu cầu tối thiểu 3 mẫu metric liên tiếp mới đưa ra đánh giá tải tài nguyên?**
    Trả lời: Một hoặc hai mẫu đo cao có thể chỉ là hiện tượng đột biến nhất thời (*Spike*) khi mở ứng dụng. Cần tối thiểu 3 mẫu liên tiếp (kéo dài khoảng 30 giây) để khẳng định tải cao kéo dài (*Sustained Overload*), tránh đưa ra các khuyến nghị kiểm tra không cần thiết.

14. **Hãy phân biệt rõ 3 tầng trạng thái: Kết nối, Điều kiện cảnh báo và Quy trình sự cố?**
    Trả lời: Kết nối là trạng thái tầng mạng (Connected/Disconnected qua socket/nhịp tim). Điều kiện cảnh báo là trạng thái ngưỡng tài nguyên (Triggered/Recovered theo Hysteresis của RuleEngine). Quy trình sự cố là trạng thái nghiệp vụ trong CSDL (Open/Acknowledged/In_Progress/Resolved do con người thao tác).

15. **Khi điều kiện cảnh báo chuyển sang RECOVERED, sự cố trên Admin có tự đóng thành RESOLVED không?**
    Trả lời: Tuyệt đối không. Điều kiện `RECOVERED` chỉ báo hiệu tài nguyên đã hạ nhiệt an toàn. Sự cố trên Admin vẫn giữ nguyên ở quy trình (`OPEN` hoặc `IN_PROGRESS`) để người quản trị kiểm tra nguyên nhân gốc rễ và chủ động bấm nút đóng sự cố.

16. **Nguyên tắc xếp hạng các bước kiểm tra (`RecommendedChecks`) được xây dựng theo tiêu chí nào?**
    Trả lời: Ưu tiên bước không xâm lấn (*Non-invasive*) lên đầu tiên (hỏi người dùng, xem Task Manager, kiểm tra ping); kế tiếp là bước tác động thấp (kiểm tra service, cáp mạng); cuối cùng mới đến can thiệp xâm lấn cao (buộc dừng tiến trình, khởi động lại máy).

17. **Tại sao thư viện không được phép đưa ra điểm số phần trăm tin cậy võ đoán?**
    Trả lời: Vì không có mô hình xác suất thực nghiệm được kiểm chứng khoa học phía sau để bảo đảm tính chuẩn xác của các con số phần trăm. Hệ thống dùng kết luận logic nhị phân rõ ràng gắn liền với bằng chứng định lượng cụ thể.

18. **Sáu kịch bản kiểm thử (Ca 1 đến Ca 6) đóng vai trò gì trong việc tích hợp nhóm?**
    Trả lời: Đóng vai trò bộ kiểm thử biên chuẩn hóa để đối soát hợp đồng giữa thư viện của D,

dữ liệu Central của Quang (B) và giao diện Admin của Vy (C), bảo đảm hệ thống phản ứng chính xác trong mọi tình huống thực tế.

19. **Sự khác biệt cốt tử giữa L4 TCP Gateway và L7 Collector trong nghiên cứu mở rộng P1 là gì?**
Trả lời: L4 Gateway chuyển tiếp luồng byte thuần túy ở tầng giao vận mà không đọc nội dung; L7 Collector giải mã từng khung tin JSON ở tầng ứng dụng, gom cụm nhiều số liệu nhỏ thành khối lớn để tối ưu hóa lưu lượng mạng WAN.

20. **Bộ giả lập Simulator 50 máy ảo trong Phụ lục P1 giúp ích gì cho đồ án?**
Trả lời: Giúp nhóm có thể mô phỏng tải của hàng chục máy trạm đồng thời trên một máy phát triển duy nhất thông qua cơ chế bất đồng bộ `Task.Run`, hỗ trợ kiểm tra năng lực chịu tải của máy chủ mà không cần 50 máy tính thật.

Phụ lục: Nghiên cứu mở rộng P1 tùy chọn (Gateway, Collector & Simulator)

TÓM TẮT 30 GIÂY

Phần phụ lục này lưu trữ các kiến thức nghiên cứu mở rộng về mặt kỹ thuật mạng quy mô lớn. Các nội dung này là **nghiên cứu tùy chọn cấp độ P1**, không nằm trong cam kết bắt buộc của Thành viên D để hoàn thành đồ án cốt lõi.

Phụ lục A: Cân bằng tải tầng 4 (Gateway L4) và Gom cụm thông điệp (Collector L7)

Khi quy mô phòng máy mở rộng lên hàng nghìn máy trạm phân bố qua nhiều lớp mạng router/NAT:

- **Gateway tầng giao vận (L4 TCP Load Balancer):** Hoạt động như một bộ chuyển tiếp kết nối thô. Gateway tiếp nhận kết nối TCP từ Agent và phân phối tới các nút Central Server theo giải thuật *Least-Connections*. Điểm mạnh là thông lượng cực cao do không phải giải mã nội dung gói tin; điểm yếu là không thể định tuyến dựa trên nội dung loại thông điệp.
- **Collector tầng ứng dụng (L7 Message Multiplexer):** Đóng vai trò trạm gom dữ liệu trung gian trong từng phòng học. Collector tiếp nhận hàng chục kết nối Agent, giải mã thông điệp qua `FrameCodec`, đưa vào hàng đệm `BoundedChannel`, sau đó gom thành từng gói dữ liệu đo đạc lớn (*Batch Multiplexing*) chuyển tiếp về Central qua một kết nối WAN duy nhất. Cơ chế này giảm đáng kể chi phí mở kết nối đồng thời trên máy chủ trung tâm.

Phụ lục B: Kỹ thuật giả lập tải phòng máy quy mô lớn (LabGuard.Simulator)

Dự án `LabGuard.Simulator` là công cụ hỗ trợ kiểm thử tải dự kiến triển khai sau khi MVP Core hoàn tất:

- **Mô hình VirtualAgent nhẹ nhàng:** Thay vì khởi tạo 50 Thread hệ điều hành gây lãng phí bộ nhớ và nghẽn CPU do chuyển ngữ cảnh (*Context Switching*), Simulator tận dụng cơ chế I/O bất đồng bộ (`async/await`) kết hợp với `PeriodicTimer(5s)` để vận hành 50 máy ảo nhẹ trên chỉ vài luồng của hệ thống.
- **Cơ chế bơm lỗi chủ động (Fault Injection):** Tích hợp bàn phím điều khiển nhằm chủ động tạo các tình huống bất thường:
  - Bơm 3 mẫu CPU 95% cho một máy ảo cụ thể để kiểm tra thuật toán phát hiện sự cố của Central.
  - Bơm chuỗi CPU hạ nhiệt $\leq 75\%$ để kiểm chứng điều kiện phục hồi Hysteresis.
  - Ngắt kết nối socket đột ngột của một máy ảo để kiểm tra cơ chế cảnh báo mất nhịp tim > 20s trên Admin Dashboard.

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN D —